LedgerNet — Team Roles and Work Breakdown (Plain-Language Guide)

Oct 30, 2026 · @Anushka

The big picture

LedgerNet is a shared bank ledger kept by five computers instead of one bank server, and all five must always agree on every balance. This guide explains, without code, what each of us builds, in what order, and where our parts touch. The technical details (exact formats, interfaces, file layouts) live in implementation.md; this is the "why and what" companion to it.

The core problem in one example

Alice has Rs 500. At the same moment she sends Rs 500 to Bob through computer 1 and Rs 500 to Carol through computer 4. If each computer decided on its own, both payments could go through and Rs 500 would appear out of nowhere. Our system must make exactly one of them succeed, and every computer must agree on which one.

The idea that solves it

All five computers keep the same numbered list of payments, called the **log**. Nobody changes a balance directly. Instead, a payment is first written into the log at a fixed position, and only then does every computer process the log from top to bottom using identical rules. Same list plus same rules means the same balances everywhere. In Alice's case, whichever payment lands higher in the list wins, and the other is rejected on every computer.

The hard part is getting five computers to agree on that list when messages get lost or delayed and computers crash. That is what **Raft** does: the computers elect one **leader**, the leader decides the order, and a payment counts as final ("committed") only once at least 3 of the 5 computers have saved it to disk. With 5 computers we can survive 2 crashing at once.

Five words we will keep using

- **Log:** the agreed, numbered list of payments. Every computer has a copy.

- **Leader and followers:** one computer orders payments; the others copy its log. If the leader dies, the rest elect a new one.
- **Committed:** saved on a majority (3 of 5), so it can never be lost or undone.
- **Nonce:** a counter on each account (1, 2, 3, …). Every payment must carry the next number, so the same signed payment can never be used twice.
- **Snapshot:** a saved copy of all balances at some point, so a restarting computer does not have to re-read the entire history.

Four promises the finished system must keep

- **No double spending:** payments that together overdraw an account never both go through.
- **Money is conserved:** the total of all balances always equals the money created at the start.
- **Agreement:** all five computers hold the same log and the same balances.
- **Durability:** once a payment is reported as done, crashing any 2 computers cannot erase it.

How the work is divided

We split along the three natural layers of the system: P decides **the order** of payments, A decides **what a payment means and how it is saved**, and V **moves messages between computers and people** and proves the whole thing works.

Member	Area	One-line role	Think of them as
P — Pranjal	Consensus (Raft)	Makes five computers agree on one list of payments, even through crashes and network splits	The referee who decides the order
A — Anushka	Ledger, security and storage	Decides whether each payment is valid, updates balances, signs and checks payments, and saves everything safely to disk	The accountant and the record-keeper

Member	Area	One-line role	Think of them as
V — Vansh	Networking, running system and testing	Connects the computers, builds the wallet people use, launches the cluster, breaks it on purpose and measures it	The builder of the roads, the shop counter and the crash-test lab

Why this split lets us work in parallel

Each layer only talks to the others through a few fixed "plugs" that we agreed on before writing anything (they are written out in implementation.md, Section 3). Think of them like the standard shape of an electric socket: as long as everyone's plug fits the socket, it does not matter how the appliance works inside.

For every plug, we also have a simple **stand-in** (a "fake") that behaves correctly but is not the real thing. So:

- P tests Raft using a fake disk, a fake ledger and a pretend network that lives inside one program.
- A tests the ledger and storage directly, with no network and no Raft at all.
- V builds the whole client-facing system on top of a fake "one-computer Raft" that approves every payment instantly.

Nobody waits for anybody. In integration week we unplug the fakes one by one and plug in the real parts.

Ground rules that keep us independent

- Each person only edits their own folders.
- The shared plugs never change silently. Any change needs a short proposal and a yes from the other two.
- Before integration week, never use a teammate's unfinished code; use the stand-in instead.

Open decisions

Four design decisions are written into the plan but still need the whole team's agreement: two on Day 1 and two during Week 1. Owners are suggestions; reassign them at the first sync and update the status as each one is settled.

#	Decision	What to settle	Suggested owner	Decide by	Status
1	Scope	Fixed 5 nodes (no adding or removing), crash faults only, money created only at the start, no fees	All	Day 1	Open
2	Insufficient-funds rule	Decided: a payment rejected for lack of money still uses up its nonce. A failed payment can never be saved and completed later, even after the account gets funds; the sender must sign a new payment with the next nonce	A	Day 1	Decided
3	Balance reads	Balance checks read a node's local copy and may be a moment behind; fully up-to-date reads are a stretch goal	P	Week 1	Open
4	Tamper-proof claim	With crash fault tolerance the hash chain detects a rewritten history but cannot prevent it; agree on how the report words this	P and A	Week 1	Open

P (Pranjal) — Consensus with Raft

P builds the engine that makes five computers agree on one ordered list of payments, no matter which of them crash or which messages get lost. P never looks inside a payment: to Raft a payment is just a sealed envelope that must be put in the same slot of the list on every computer.

What P's part must do

- **Elect a leader.** When no leader is heard from for a short random time, a computer asks the others to vote for it. Whoever gets 3 of 5 votes leads. Only a computer with an up-to-date list may win, so an old, out-of-date computer can never become leader and erase newer payments.

- **Copy the list to everyone.** The leader adds each new payment to the end of its list and sends it to the followers. Followers check that their list matches the leader's up to that point; if not, the leader walks back until they match and then fixes the follower's copy.
- **Decide when a payment is final.** A payment is committed once 3 of 5 computers have saved it to disk. Only then is anything told to the client.
- **Hand committed payments to the ledger, in order.** P passes each committed entry to A's ledger one at a time, in list order, and tells the waiting client the result.
- **Remember things across crashes.** The current election round and who it voted for are saved to disk before answering anyone, so a restarted computer can never vote twice or forget a promise.
- **Recover after a restart.** Load the latest snapshot, replay the saved payments after it, then fetch whatever was missed from the leader.
- **Take snapshots and trim the list.** Every 1,000 committed payments, ask the ledger for a snapshot, save it, and throw away the old part of the list. If a follower has fallen so far behind that the leader already threw away what it needs, the leader sends the whole snapshot instead.
- **Behave well in a network split.** A leader that cannot reach a majority steps down on its own (so the smaller side refuses new payments). A cut-off computer first asks "would you vote for me?" before starting an election, so it cannot disrupt the cluster when it reconnects.

Step by step

1. **Build a pretend network inside one program.** Five Raft computers run in one process and "send" messages to each other through code that can drop, delay, duplicate or block messages on purpose, and can crash and restart a computer while keeping its "disk". This is P's test lab and makes bugs reproducible.
2. **Leader election.** Get one stable leader among 3 and then 5 computers, and a new one when the leader is killed.
3. **Copying the list and committing.** Leader sends entries, followers check and fix mismatches, the leader counts saved copies and commits.
4. **Applying in order and answering clients.** Pass committed entries to the ledger and let the waiting request know if its payment made it, or if it was lost because leadership changed.
5. **Saving to disk and restarting.** Make every promise durable before replying, and rebuild correctly after a crash.

6. **Snapshots and catching up far-behind computers.**
7. **Split-network protections** (stepping down without a majority, and asking before starting an election).
8. **Heavy random testing.** Run hundreds of randomized runs with crashes and network chaos, and after every step check the safety rules: never two leaders in the same round, every computer's committed list identical, nothing committed ever disappears.
9. **Stretch goal:** reads that are guaranteed to see the very latest committed balance.

What P depends on

In theory P needs three things from teammates: a disk to save the list on (from A), a ledger to apply payments to (from A), and a way to send messages between real computers (from V). In practice P needs **none of them until integration week**, because the stand-ins (a memory-only disk, a simple recording ledger and P's own pretend network) behave exactly like the real ones from Raft's point of view. The pretend network is actually better for testing than the real one, because P controls every lost message.

Done looks like

All randomized tests pass across hundreds of runs, the Raft code uses only the agreed plugs, and P has written a short note explaining how each safety rule is guaranteed (useful for the report).

A (Anushka) — Ledger, security and storage

A builds everything that gives a payment its meaning and keeps it safe: the signature checks, the rules that accept or reject a payment, the account balances, the tamper-evident chain of hashes, and the files on disk that survive a crash. A also builds the tools that prove the system is correct and that turn our measurements into graphs.

What A's part must do

- **Payments and signatures.** Each person has a private key and a public key. A payment says "from this account, to that account, this many paise, with this nonce" and is signed with the sender's private key. Anyone can check the signature with the public key, but nobody can forge it. A owns this format and its tests.

- **The ledger rules.** For every committed payment, in order, every computer runs the same checks: is it well formed, is the amount above zero and to a different person, is the signature real, is the nonce the next expected one, is there enough money. Then it updates balances or rejects the payment. These rules must give identical answers on every computer, every time (no randomness, no clocks, no decimals).
- **Account storage.** Balances, nonces and payment receipts are kept in a small SQLite database on each computer.
- **The hash chain.** Each processed entry is mixed into a running fingerprint (SHA-256) together with the previous fingerprint. If any computer's history differed by even one byte, its fingerprint would differ, so comparing fingerprints instantly shows whether all five agree.
- **The write-ahead log on disk.** Raft's list of payments is written to a file and forced onto the physical disk before Raft is allowed to say "saved". After a crash, half-written records at the end are detected (using checksums) and cleanly cut off.
- **Snapshots.** Save a full copy of all balances safely (write to a temporary file, then swap it in), keep the last couple, and load the newest good one on restart.
- **The auditor.** A separate tool that reads each computer's files after a test and independently re-checks everything: the hash chain is unbroken, money is conserved at every step, nonces only go up, and all five computers' histories match.
- **Load generator.** A tool that fires many signed payments at the cluster at a chosen rate and records how long each one took.
- **Analysis and graphs.** Scripts that turn the recorded numbers into latency, failover and recovery charts for the report.

Step by step

1. **Lock down the payment format.** Write tests that a payment survives encoding and decoding, and that changing any single byte breaks the signature. Save known example payments so an accidental format change is caught immediately.
2. **Build the real ledger on SQLite.** Implement the rules exactly as written in the shared specification.
3. **Prove it matches the specification.** We already have a simple, obviously-correct reference ledger. Feed both ledgers thousands of random payment streams (double spends, replays, forged signatures, garbage, overdrafts) and check they always give identical results and identical fingerprints. This is A's most important test.

4. **Ledger snapshots.** Save and restore the database, and check that continuing after a restore gives the same result as never stopping.
5. **Snapshot files on disk**, with corruption detection and fallback to the previous snapshot.
6. **The write-ahead log**, including crash tests that cut the file at every possible byte and check that exactly the fully written part survives.
7. **The auditor**, tested by deliberately corrupting a file and checking that it is caught.
8. **Graph scripts**, developed early on made-up data in the agreed spreadsheet format, so they are ready the moment real data exists.
9. **Load generator** (Week 2), then real measurements and final graphs in Weeks 4–5.

What A depends on

A's core work (format, ledger, storage, snapshots, auditor) depends on **nobody**. It can be built and tested with no network and no Raft: A just feeds payments directly into the ledger and writes entries directly into the log file.

There are two small dependencies, both with workarounds:

- **The load generator** is easiest to build on top of V's client library (the part that finds the leader and retries). V promises it by the end of Week 1. Until then, A can test against a tiny stand-in server that A writes in an hour.
- **The graphs** need real measurements, which only exist once the full system runs (Week 4). Until then A works on made-up data in exactly the same format.

In the other direction, two teammates rely on A's output later: P's Raft will save its list using A's write-ahead log, and V's end-to-end tests will run A's auditor after every scenario.

Done looks like

The real ledger matches the reference on tens of thousands of random cases, the disk files survive every simulated crash, the auditor catches deliberate corruption, and the graph scripts produce figures.

V (Vansh) — Networking, the running system and testing

V turns the pieces into five real programs talking over a real network, gives people a wallet to use them, and then runs the experiments that break the system on purpose and measure it. V is also the one who drives the integration week, because V's node program is where everyone's parts get plugged together.

What V's part must do

- **The message format between computers.** Using Protocol Buffers and gRPC, define and implement the actual messages Raft sends (votes, list updates, snapshots) and the messages clients send (submit payment, check balance, check receipt, check status).
- **A translator** between the network message format and the plain Python objects Raft uses, so P never has to think about networking.
- **Fault injection.** Built into the networking layer: on command, drop a share of messages, delay them, send duplicates, or cut off chosen computers completely (a network split). This is controlled at runtime through a small admin service.
- **The client-facing service.** Receives a payment, quickly rejects obvious garbage (bad format or fake signature), asks Raft to add it, waits for the result and replies. If this computer is not the leader, it replies "not the leader, try that one".
- **The node program.** The one program that starts a computer: reads the config, builds the disk, the ledger, the network and Raft, and starts serving. It has switches to use either the real part or the stand-in for each component, which is what makes integration safe.
- **The client library and wallet.** The wallet creates keys, sets up the starting balances, sends payments, shows balances and shows the status of all five computers. Underneath, the client library finds the leader, follows "try that one" hints, retries safely, and resolves "did my payment go through?" after a timeout by checking receipts and nonces.
- **The cluster launcher.** Starts, kills (hard, like a power cut), restarts and wipes nodes, either on one laptop or across RCE machines over SSH, and can split the network and wait until all nodes agree again.
- **End-to-end scenarios.** The full demos: double spend, leader crash under load, network split, replayed and forged payments, unreliable network, crashes of 2 computers, full restart, a far-behind computer catching up.

- **Experiment runs.** Scripts that run the latency, failover and recovery experiments and save the results in the agreed format for A's graphs.

Step by step

1. **Generate the network code and the translator,** with tests that every message survives the round trip.
2. **The client-facing service on the stand-ins.** Run it on top of the "one-computer Raft" stand-in and the reference ledger, and submit a real signed payment end to end. This works on Day 1 with nothing from P or A.
3. **Client library** (promised to A by the end of Week 1) and **wallet**.
4. **Fault injection and the real networking layer for Raft,** tested with two programs on one laptop.
5. **The node program with component switches.**
6. **The cluster launcher,** first local, then RCE (keep data on each machine's local disk, not the shared network drive, because saving to a network drive is slow and would ruin our timing numbers; pick ports that will not clash with other students; check the machines' clocks agree).
7. **End-to-end scenarios,** first rehearsed on the single-computer stand-in to debug the test scripts themselves, then for real after integration.
8. **Experiment scripts** and runs on laptops and RCE.

What V depends on

V's part is the place where all three parts meet, so V has the most theoretical dependencies, but none of them block V before integration:

- **Raft (from P):** replaced by the "one-computer Raft" stand-in that approves every payment instantly. The client service, wallet and launcher can all be built and tested on it.
- **The ledger (from A):** replaced by the reference ledger, which follows exactly the same rules.
- **Disk storage (from A):** replaced by memory-only storage via a switch.
- **The auditor (from A):** until it exists, V checks agreement by comparing the fingerprints each node reports.

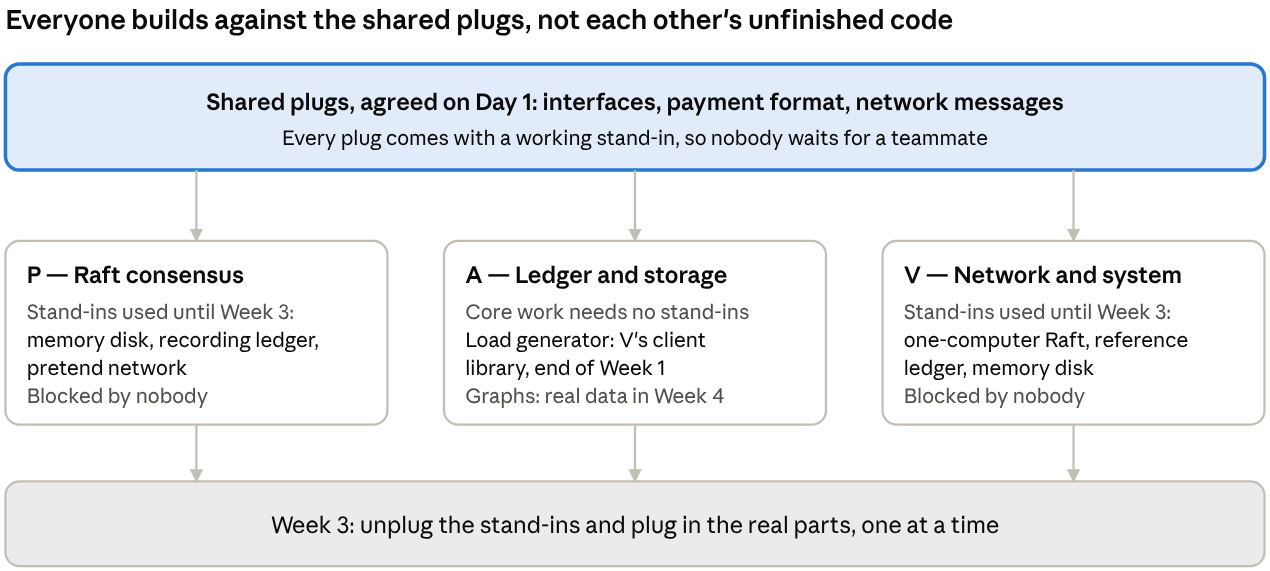
In the other direction, A's load generator uses V's client library (end of Week 1), and the whole team relies on V's launcher and node program during integration and experiments.

Done looks like

A single node with all stand-ins serves the wallet end to end, the networking and fault-injection tests pass, the launcher can start and kill five local programs, and after integration all demo scenarios pass on laptops and RCE.

Interdependencies in detail

In the finished system the three parts depend on each other completely, but while building them only one small thing has to arrive on time: V's client library for A, by the end of Week 1. Everything else is covered by stand-ins until integration in Week 3.



how the three parts connect · shared plugs, stand-ins, integration

Each of us plugs into the agreed plugs at the top, uses stand-ins for everyone else's part, and only meets the real parts in Week 3.

How the finished system depends on itself

It helps to follow one payment through the finished system to see where the hand-offs are:

1. A person uses V's **wallet**, which uses A's **signing rules** to sign the payment.
2. V's **network service** receives it, uses A's rules to reject an obvious fake, and hands it to P's **Raft**.

3. P's Raft saves it using A's **write-ahead log** and sends it to the other computers through V's **network layer**.
4. Once 3 of 5 have saved it, P's Raft gives it to A's **ledger**, which checks it and updates balances.
5. V's service reports the result back to the wallet.

So every payment passes through all three people's work. That is exactly why we agreed the plugs first: each hand-off above is one of the plugs, and each one has a stand-in.

Every dependency, who needs what and when

Who needs it	What they need	From	Why, in simple words	Real version arrives	Stand-in until then
A	Client library (finds the leader, retries safely)	V	The load generator must send thousands of payments to whichever computer is leader	End of Week 1	A tiny one-computer test server A writes in about an hour
A	Real measurements	V's experiments	Graphs need real timings	Week 4	Made-up data in the same spreadsheet format
P	Disk storage for the list and snapshots	A	Raft must not reply "saved" until it really is on disk	Week 3	Memory-only disk that survives a pretend crash
P	A ledger to apply payments to	A	Raft hands over committed entries and must not care what is inside	Week 3	A recording ledger that just keeps a fingerprint
P	Network between real computers	V	Raft's messages must travel between machines	Week 3	P's own pretend network (better for testing)
V	Raft	P	Payments need to be ordered and replicated	Week 3	One-computer Raft that approves instantly
V	The real ledger and storage	A	Real balances and real crash safety	Week 3	The reference ledger and memory-only disk
V	The auditor	A	Independent correctness check after each demo	Week 2–3	Comparing the fingerprints each node reports

Hidden dependencies (agreements, not code)

Some dependencies are not about one person waiting for another person's code, but about everyone respecting the same agreement. These are the ones that cause bugs if someone forgets:

- **The payment format and ledger rules (A owns, everyone uses).** V's wallet builds payments and the load generator signs them, so if A changes the format, the wallet and load generator break. Any change goes through the team.
- **Determinism (A and P).** The ledger must give identical results on every computer: no randomness, no clock readings, no decimals, always sorted. If A broke this, P's Raft would be perfect and the computers would still disagree.
- **Save-then-trim order for snapshots (P and A).** A's snapshot must be safely on disk before P throws away the old part of the list, otherwise a crash at the wrong moment loses data. Both must follow this order.
- **"Saved" really means saved (P and A).** P relies on A's write-ahead log being on the physical disk before it returns. If A skipped this to go faster, Raft's durability promise would silently break.
- **Event names for measurements (P, V and A).** P's Raft writes events like "leader elected" and "snapshot loaded" with exact names. V's experiments collect them and A's graphs read them. All three must use the same names and spreadsheet columns, which are fixed in implementation.md.

How we avoid getting blocked

- The plugs and stand-ins are written out completely before anyone starts, so everyone can begin on Day 1.
- The one early delivery (V's client library) is small and has a backup plan.
- Any change to a shared plug needs the other two to agree, with the stand-ins updated at the same time.
- A 30-minute weekly sync to show progress and raise plug changes early.

Working rules everyone follows

Integration goes smoothly only if all three parts look and behave as if one person wrote them. These rules fix the choices that would otherwise differ between us: tools, versions, units, naming, error handling, logging, testing and git. If a rule gets in the way, raise it at the weekly sync and change it for everyone, never just for yourself.

1. Same tools, same versions

- **Python 3.10** on every laptop and on RCE (check with `python --version`). Use a virtual environment per checkout; never install packages globally.

- **One shared dependency list** (`pyproject.toml`) with **exact versions pinned**. Nobody adds a new library without telling the other two first; if it is added, it goes in that file in the same pull request.
- **Formatter and linter:** `black` for formatting and `ruff` for linting, both run before every commit. This removes all arguments about style and keeps diffs clean.
- **Type hints** on every function and method that another file uses. The shared plugs are typed, so our code should match them.
- **Protocol Buffers:** the network code is always generated with `make proto` from the one shared `.proto` file. Never edit generated files and never commit them.
- Everything runs on **Linux** (RCE). If you develop on Windows or macOS, test on RCE or a Linux machine at least once a week, because file-saving behaviour differs between systems.

2. Same units and data conventions

Most integration bugs are two people meaning different things by the same number. These are fixed for everyone:

Thing	Always	Never
Money	Whole numbers of **paise** (Rs 500 = 50000)	Rupees with decimals, or any float. Rupees appear only on screen in the wallet
Account and keys inside code	Raw 32-byte values	Hex strings. Hex is used only in files, logs and on screen
Hashes shown or stored as text	Lowercase hex	Uppercase or base64
Log positions (indexes) and election rounds (terms)	Start at 1; 0 means "none yet"	Starting at 0
Durations in names	Unit in the name: `timeout_ms` , `elapsed_s`	Bare `timeout`
Measuring time spent	`time.monotonic()`	`time.time()` , which jumps when the clock is adjusted
Timestamps in logs and spreadsheets	Unix seconds (`time.time()`)	Formatted date strings

Thing	Always	Never
Node names	`n1` to `n5`	`node1`, `1`, host names
Numbers packed into bytes	Big-endian	Little-endian or platform default

3. Same naming and structure

- Python's usual style: `snake_case` for functions, variables and files; `CamelCase` for classes; `UPPER_CASE` for constants.
- Use the words from the shared glossary everywhere: *log*, *entry*, *index*, *term*, *commit*, *apply*, *snapshot*, *transfer*, *nonce*, *receipt*. Do not invent synonyms like *tx list*, *block number* or *round*.
- Each person works only in their own folders. Shared plugs live in the common folder and change only through the team process.
- No hard-coded ports, paths, timeouts or node lists in the code. They come from the config file or the Raft settings object, so the same code runs on a laptop and on RCE.

4. Same way of handling time, concurrency and failure

- **Everything is asyncio.** Never use blocking calls inside the running program: no `time.sleep` (use `asyncio.sleep`), no blocking network libraries, no extra threads unless the team agrees.
- The ledger and storage are only ever called from the event-loop thread, so nobody needs locks. Do not add threads that touch them.
- **Use the shared error types.** A failed network call is always `TransportError`; "I am not the leader" is always `NotLeaderError`. Never return `None` or `False` to mean "it failed" unless the shared plug says so.
- **Never swallow an exception silently.** Either handle it on purpose and log it, or let it rise.
- Every network call has a timeout. Nothing in the system may wait forever.

5. Determinism rules for anything that touches balances

These apply to A's ledger and to anything P passes into it. Break one and the five computers will drift apart even with perfect Raft.

- No random numbers, no reading the clock, no reading environment variables, no floats.

- Never rely on the order of a dictionary or set when hashing or deciding anything; always sort first.
- The same input must always give the same output, byte for byte, on every machine.

6. Same logging and events

- Use Python's `logging`, never `print`. Every line starts with the node name and current term so logs from five nodes can be merged and read together.
- Levels: `DEBUG` for every message sent (off by default), `INFO` for role changes, snapshots and commit rate, `WARNING` for retries and timeouts, `ERROR` for anything that should never happen.
- Measurement events (leader elected, snapshot loaded and so on) use **exactly** the names and fields listed in implementation.md. P writes them, V collects them, A graphs them, so a renamed event silently breaks the graphs.

7. Same way of testing

- `pytest` with `pytest-asyncio` for everything; tests live in the `tests/` folder matching your area.
- Every random test uses a **fixed seed and prints it** when it fails, so anyone can rerun the exact failure.
- The shared contract tests must pass in every pull request. Your code must pass them, not just your own tests.
- Every bug found gets a test that reproduces it before it is fixed.
- The full quick test suite should finish in under 5 minutes so everyone actually runs it; long fuzz runs go behind a separate flag.

8. Same git workflow

- Nobody pushes directly to `main`. Work on branches named `p/<topic>`, `a/<topic>`, `v/<topic>`.
- Small pull requests (ideally under 400 lines), each reviewed by one of the other two. Reviewers check the rules on this page, not only whether the code works.
- Pull the latest `main` into your branch at least once a day, so differences surface early instead of in Week 3.
- Commit messages say what changed and why, in one line: "raft: step down when no majority is heard for one election timeout".

- Never commit data folders, generated code, real private keys or results files over a few MB. Test keys for the demo accounts are fine.
- The automatic test run must be green before merging.

9. Changing a shared plug

1. Write a short proposal in the team chat: what changes, why, and who is affected.
2. Both other members agree.
3. One pull request, titled `CONTRACT: ...`, changes the plug, the stand-ins and the contract tests together.
4. Everyone pulls it the same day.

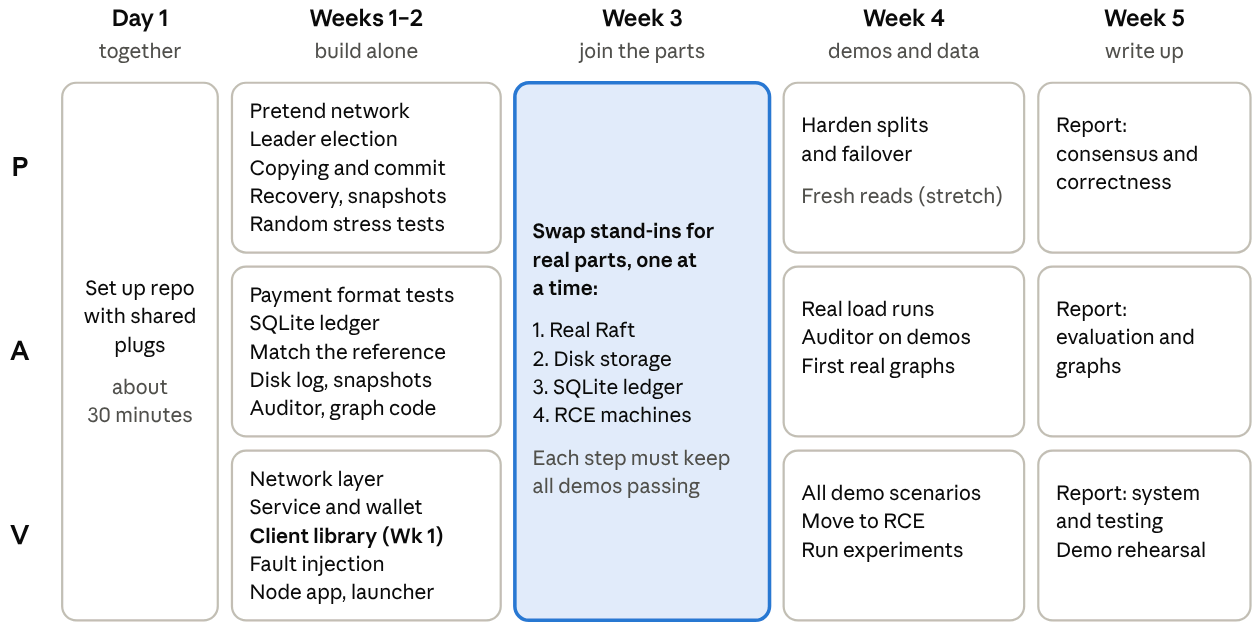
10. Checklist before opening a pull request

- [ ] `black` and `ruff` pass
- [ ] All tests pass locally, including the shared contract tests
- [ ] No floats or rupees in money code; units are in variable names
- [ ] No hard-coded ports, paths or timeouts
- [ ] No blocking calls inside async code; every network call has a timeout
- [ ] Errors use the shared error types and none are silently ignored
- [ ] Logging uses `logging` with node and term; event names match implementation.md
- [ ] No new library added without the team agreeing
- [ ] A short note in your area's README if you made a design decision others should know

Timeline and joining the pieces together

We spend Weeks 1–2 building alone on stand-ins, Week 3 joining the real parts, and Weeks 4–5 on demos, measurements and the report. This assumes about five weeks; stretch or squeeze each phase evenly to fit our real deadline.

Two weeks of solo work, one week to join, two weeks to prove and report



five-week roadmap · a lane per member

The only hard date inside the solo phase is V's client library at the end of Week 1; Week 3 is the one week where all three must work closely together.

Integration week, step by step

We swap one stand-in for the real part at a time. After each swap, all the demo scenarios must still pass before we do the next one. If something breaks, we know exactly which new part (or which hand-off) caused it.

1. **Real Raft over the real network (P with V).** Five real programs, still using memory-only storage and the reference ledger. This tests only Raft and the networking together.
2. **Real disk storage (P with A, V running it).** Switch to A's write-ahead log and snapshot files. Now crash-and-restart tests really mean something.
3. **Real SQLite ledger (A with V).** Switch to A's ledger. A's auditor must pass after every scenario.
4. **Move to RCE machines (V, everyone helping).** The same tests, now across separate machines.

What each person must have ready before Week 3

- **P:** Raft passing all randomized tests on the pretend network.

- **A:** ledger matching the reference, disk log and snapshots passing crash tests, auditor working.
- **V:** node program with the stand-in switches, launcher starting and killing five local programs, wallet working on the stand-ins.

Demos, measurements and how we prove it works

Every demo ends the same way: all five computers show the same fingerprint, total money equals the starting total, and A's auditor says PASS. V runs the scenarios; P and A explain what the audience is seeing in their parts.

Demo	What we do	What should happen	Promise it proves	Main explainer
Double spend	Alice (Rs 500) sends Rs 500 to Bob and Rs 500 to Carol through two different computers at the same instant	Exactly one succeeds, the same one on every computer	No double spending, agreement	A
Leader crash under load	Steady stream of payments, then kill the leader	A new leader takes over within about a second, payments resume, the old leader rejoins and catches up	Durability, agreement	P
Network split	Cut the network into 3 and 2 computers	The 2-side refuses new payments, the 3-side keeps going, everyone agrees after reconnecting	Safety in a split	P
Far-behind computer	Turn one off, process 2,000+ payments, turn it back on	It receives a snapshot and catches up	Recovery	P and A

Demo	What we do	What should happen	Promise it proves	Main explainer
Replay and forgery	Resend an old signed payment; change one byte of a signature	Both rejected identically on all computers	Security	A
Bad network	Drop 20% of messages, delay and duplicate others	Slower, but still correct	Agreement	V
Crashes	Crash 2 computers, then everyone at once, then restart	No confirmed payment is ever lost	Durability	V

What we measure (A makes the graphs, V runs the experiments)

- **Commit latency:** how long a person waits from pressing "send" to seeing "done", at different loads, on laptops and on RCE.
- **Failover time:** after the leader dies, how long until a new leader exists and how long until payments flow again.
- **Recovery time:** how long a restarted computer takes to be fully up to date, comparing "re-read the whole history" against "load a snapshot, then read only what came after", for histories of 10,000, 50,000 and 100,000 payments.